

Experiment 2

Module: Automation with Selenium

Experiment: Synchronization and Explicit Waits

Problem statement: Interact with dynamic web content that loads asynchronously without using hardcoded time.sleep().

Objective: Use WebDriverWait and an expected condition to wait for an element to become visible, then validate its content.

Tools and concepts

Python; Selenium WebDriver; ChromeDriver; WebDriverWait; expected_conditions; By.CSS_SELECTOR; By.ID.

Implementation

```
from selenium import webdriver

from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import WebDriverWait

from selenium.webdriver.support import expected_conditions as EC

import time


driver = webdriver.Chrome()

driver.maximize_window()

try:

    driver.get("https://the-internet.herokuapp.com/dynamic_loading/2")

    driver.find_element(By.CSS_SELECTOR, "#start > button").click()

    wait = WebDriverWait(driver, 10)

    hello_world_element = wait.until(

        EC.visibility_of_element_located((By.ID, "finish"))

    )
```

```
result_text = hello_world_element.text
```

```
assert "Hello World!" in result_text
```

```
print(f"Test Passed: Extracted text is '{result_text}'")
```

finally:

```
time.sleep(4)
```

```
driver.quit()
```

Test execution and result

The rewritten flow was tested against the target page. The Start button was activated, the loading state was observed, and the explicit visibility wait completed successfully.

Check	Observed result
Target page	https://the-internet.herokuapp.com/dynamic_loading/2
Synchronization	WebDriverWait; maximum 10 seconds
Assertion	Passed: page displayed "Hello World!"
Outcome	PASS

Dynamically Loaded Page Elements

Example 2: Element rendered after the fact

Loading... 

Powered by [Elemental Selenium](#)

Fork me on [GitHub](#)

Dynamically Loaded Page Elements

Example 2: Element rendered after the fact

Hello World!

Powered by [Elemental Selenium](#)

Fork me on [GitHub](#)

Conclusion: Explicit waits synchronize the test with the page state and avoid brittle fixed delays. The corrected indentation makes the Python implementation executable and ensures the browser is closed even if the assertion fails.